M03-L03 · Triage, en busca del culpable

Alberto Basalo

TESTING INTELIGENTE

Triage de fallos

> *Detectar el origen y aprobar la solución.*

CONNECTIONS

- La prueba espera una cosa y la app devuelve otra. ¿**Quién** se ha equivocado?
- Una prueba antigua falla y nadie la ha tocado. ¿**Quién** lo arregla?

CONCEPTS

Triage

Investigar los fallos de una spec para decidir:

- **la causa**: varios fallos pueden tener la misma.
- **el origen**: prueba, código o especificación.
- **quién corrige**: el agente responsable de ese origen.
- **qué falta saber**, si la evidencia no basta.

> *Primero decidir, después corregir.*

CONCEPTS

El síntoma no es la causa

- El error dice **dónde** se rompió la prueba, no **por qué**.
- El primer sospechoso suele ser el código.
- Las reglas de **otras specs** también cuentan.
- **Hueco**: un dato que la prueba necesita y que el `.feature` no da.

> *Busca la fuente de verdad antes de culpar a nadie.*

CONCEPTS

Una spec de corrección

Necesitamos una `fix.spec.md` con el mismo formato pero distinto propósito:

- **Problem**: las causas de los fallos con enlaces a las evidencias y los huecos de la spec.
- **Solution**: qué cambia cada origen, en orden: *spec*, *código*, *pruebas*.
- **Verification**: qué pruebas deben pasar y qué aserciones no cambian.

> *Se implementa el arreglo y se repite la verificación.*

CONCRETE PRACTICE

Los cinco fallos de L02

Prueba	Síntoma	Primer sospechoso
`S0005-R7-3`	Espera `200`, recibe `409`	El código rechaza el cambio de estado
`S0003-R7`	La fila tiene más texto	S0004 rompió la lista
`S0005-R8`	No encuentra `Passengers`	Falta la página
`S0005-R9-1`, `S0005-R9-2`	No encuentra `Login`	Falta la protección de la página

La entrada es `specs/S0005-passenger-bookings/e2e-report.md`, con sus evidencias.

CONCRETE PRACTICE

P1 · La skill `/triage-e2e-failure`

Crea la skill con la guía y la plantilla de `/review-skills`:

- Recibe **una spec** y lee todos los fallos de su `e2e-report.md`, también las regresiones.
- Lee la prueba, el Page Object, el `.feature`, `spec.md`, las specs relacionadas y el código.
- Compara **tres hipótesis** para cada fallo y agrupa los fallos por causa.
- Cada cita lleva su archivo, su línea y **el texto de la línea**. La skill lo comprueba antes de mostrarla.
- Muestra el veredicto y **para**: espera tu aprobación.
- Con tu aprobación, escribe la spec de corrección `fix.spec.md` en la carpeta de la spec.
- **No toca pruebas, código ni specs.**

CONCRETE PRACTICE

P2 · Lanzar el triage de `S0005`

En una conversación nueva, usa `/triage-e2e-failure` con la spec `S0005`.

El agente lee los cinco fallos, compara las hipótesis y muestra **tres causas**. Después para.

No apruebes todavía. Primero comprueba cada causa.

CONCRETE PRACTICE

P3 · El síntoma engaña: `S0005-R7-3`

Hipótesis	Evidencia
Código	El `409` viene de `changeStatus` (línea 24), no de la reserva.
Spec	S0004 prohíbe `planned → failed` (`spec.md:24`).
Prueba	El setup salta de `planned` a `failed` (línea 238).

El `.feature` dice «a launch has status failed» y no dice cómo se llega a `failed`. El agente lo supuso.

Origen: **la prueba**. Hueco: **el `.feature`**.

CONCRETE PRACTICE

P3 · Nadie se equivocó: `S0003–R7`

- La spec pide mostrar el cohete, la fecha y el precio (`S0003 spec.md:75`). No dice «solo».
- S0004 añade a la fila el estado y los botones, y S0005 añade el enlace. Las dos specs lo piden.
- La prueba compara **todo el texto** de la fila con `toHaveText` (línea 25).

Origen: **la prueba**. La aserción es más estricta que el criterio.

Ya lo viste en M02-L05. Entonces tocaba escribir buenas pruebas; hoy toca su triage.

> *Una prueba antigua también puede tener el error. La spec evolucionó; la prueba no.*

CONCRETE PRACTICE

P3 · Una causa, tres fallos: `S0005-R8` y `S0005-R9`

- `error-context.md` muestra `Page not found` en `/launches/:id/passengers`.
- `spec.md:54` y el enlace de la app (`launches.page.ts:26`) usan `/bookings`.
- El Page Object abre `/passengers` (`bookings.page.ts:29`).
- El `.feature` solo dice «the passenger page». Ya lo viste en L01.

Origen: **la prueba**. Hueco: **el `.feature`**.

CONCRETE PRACTICE

P4 · Aprobar y guardar el veredicto

Si cada cita dice lo que el agente afirma, aprueba. El agente escribe `fix.spec.md`.

Causa	Fallos	Origen	Corrige	Hueco
Transición no permitida	`S0005-R7-3`	Prueba	«pruebas»	`.feature`
Aserción demasiado estricta	`S0003-R7`	Prueba	«pruebas»	—
Ruta supuesta	`S0005-R8` , `R9-1` , `R9-2`	Prueba	«pruebas»	`.feature`

Cinco fallos, tres causas, un origen. **El código no tiene ningún error.**

¿Podría otro agente corregir **solo con este documento**?

CONCLUSION

Origen antes que corrección

El triage compara hipótesis con evidencias y devuelve cada fallo a su origen. El primer sospechoso no siempre es el culpable.

Antes de aceptar el diagnóstico

¿Puedes abrir **la evidencia** que descarta cada hipótesis?

Siguiente paso: caso cerrado

En L04, cada agente implementará su parte de `fix.spec.md` con su skill, sin más contexto. Una ejecución nueva de las pruebas dirá qué queda.

Alberto Basalo